Project Outline

Applied Programming | 2026-02 semester

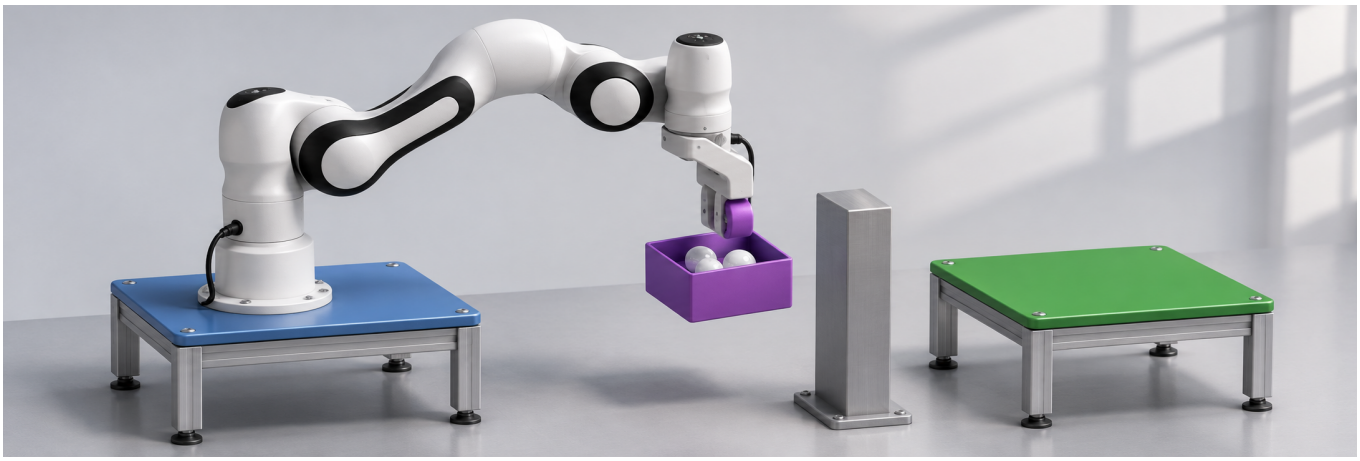


Figure 1. Franka Emika Panda sample transport concept from the original Project Outline. Concept illustration.

Project Background and Task

Physical AI connects computational decisions to physical action. Design robot placement, handoff locations, seven-joint paths and speed to transport the required samples through a materials-processing cell. Preserve the samples while reducing carrying distance and completion time.

Genesis evaluates how your motion affects the robot, container and contents. A geometrically valid path can still cause tracking error, prohibited contact or sample loss. Compare the outcome with your design choices and revise.

The current glass-bead stage has been tested on an RTX 4090. Verify your own design. Official bead competition and final selection are active. GTX 1660 SUPER runtime and memory remain unmeasured. The material release plan appears with the layouts below.

Your implementation

Implement plan_path in student/joint_strategy.py. Each request gives one arm's exact start and goal. Generate all intermediate configurations with RRT, bidirectional RRT, A*, or another suitable taught method. Return your seven-angle path in radians, or None when search fails.

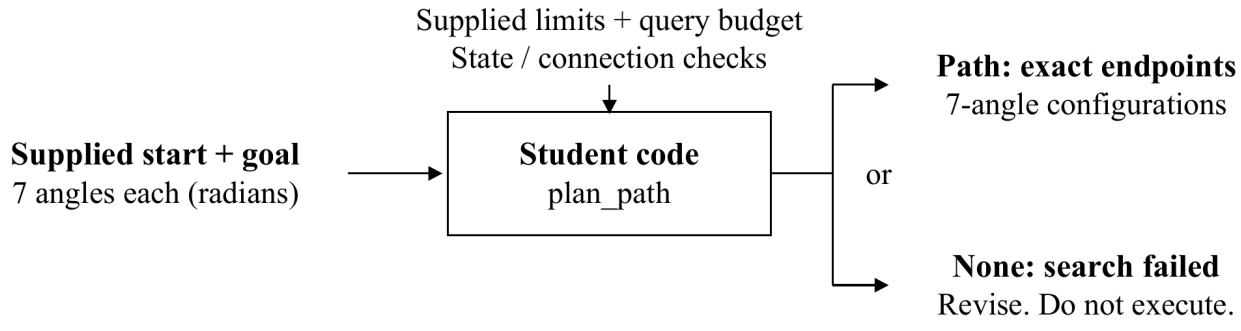


Figure 2. The student planner constructs the path between supplied endpoints. A failed search returns None for revision.

Open the project repository | Read exact API contracts

1. Provided Support and Development

Plan the seven arm joints of the Genesis Franka Emika Panda. FK/IK, automatic grasp/place and low-level control are provided. You do not implement gripping contact acquisition, a physics solver or a separate braking controller.

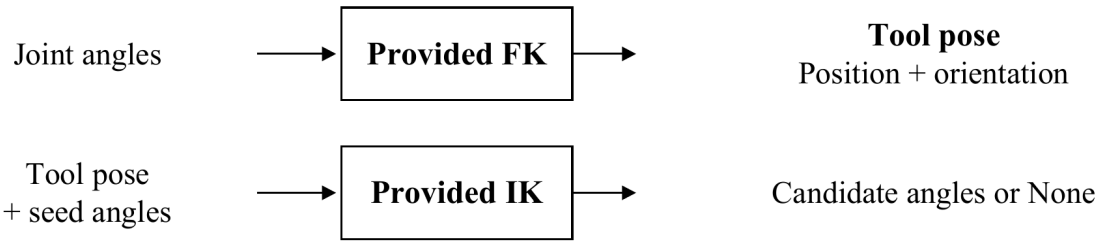


Figure 3. Supplied FK and IK map between joint angles and tool poses. They do not establish collision freedom.

Use supplied geometry and whole-robot state/edge checks to test candidate connections. Choose clearance, search resolution and timing. Final validation resolution stays fixed.

Development process

Check a CPU plan. Optionally preview it or test the same commands with an empty cup. Replay with the currently released material and inspect physical results before revising.

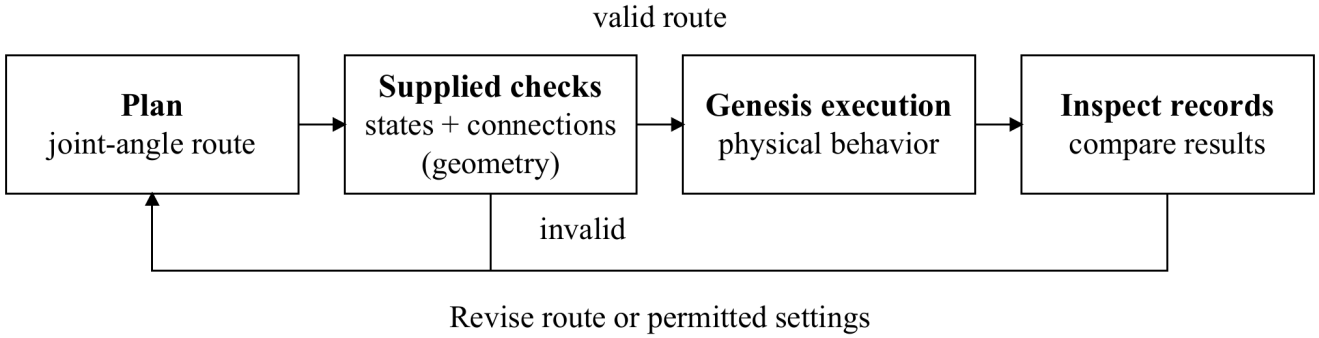


Figure 4. Geometric validation and physical execution answer different questions. Re-export after changing student inputs, then replay that exact plan.

2. Layout and Design Choices

Compare the layouts and permitted team sizes before choosing a complexity. The required carries and **simulated** deadlines below describe the current glass-bead stage. A handoff means supported placement followed by automatic acquisition of the same cup by the next arm.

Complexity / CLI	Students	Carries / handoffs	Base max	Deadline
Lower / easy	1	1 / 0	80	40 s
Moderate / medium	1-2	2 / 1	90	85 s
Higher / hard	2-3	3 / 2	100	105 s

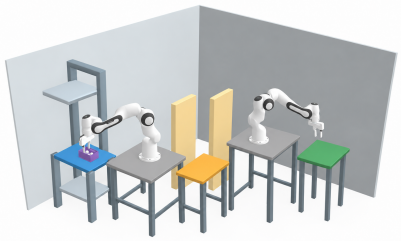
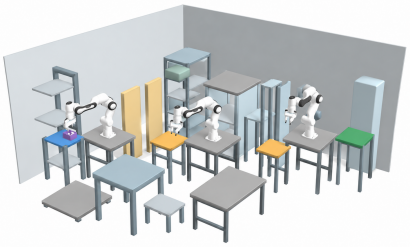
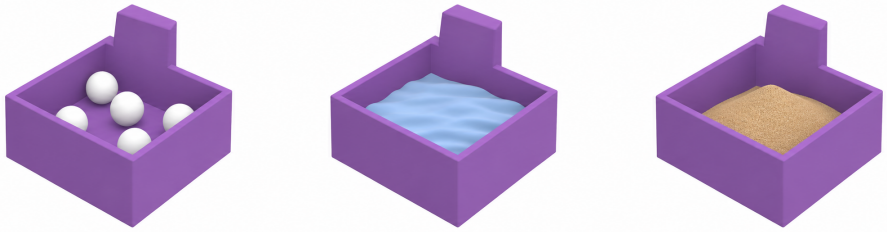
Lower complexity	Moderate complexity	Higher complexity
		

Figure 5. Layout illustrations from the original Project Outline. These are conceptual examples. Use the current map and 3D viewer for exact geometry and permitted placements.

Complete the required transport tasks for the materials released for your selected complexity. Glass beads are released first. Additional materials will be released according to students' progress.

Glass beads **Aqueous solution** **Silica sand**



Original Project Outline concept illustrations. The current glass-bead tests use four beads.

Choose the design inputs

Edit the selected **student/easy_design.json**, **student/medium_design.json** or **student/hard_design.json**. Replace editable null values with your choices. Use the published mounting and support bounds in world metres.

Student choice	Input and constraint
Robot installation	Choose mount_region, base X/Y and yaw. The chosen physical surface determines height. Arbitrary base Z is not editable.
Handoff location	Select distinct permitted internal supports and XY points. Allow the full cup footprint plus the CPU contact margin (currently 4 mm inside each support edge). Keep fixed source and final targets unchanged.
Seven-joint route	Implement plan_path in student/joint_strategy.py. Generate your own intermediate configurations and connections.
Speed	Set joint_speed_scale, or implement choose_edge_durations_s for independent edge times.

The common scene contains three robot entities. Complete all three base entries, even when only one or two arms carry the cup. Other arms remain parked collision geometry. Complete unused relay entries as valid scene inputs. They do not add transport legs. Lower fixes the first relay entry as its finish. Moderate fixes the second. Higher selects both internal relays and finishes at goal_support.

3. Construct and Check a Path

```
plan_path(request, oracle, *, kinematics=None, context=None)
```

Each request gives one arm's exact start and goal configurations, seven inclusive joint bounds, a seed and a validity-query budget. Use RRT, bidirectional RRT, A*, or another suitable taught method. You may explore by restricting selected degrees of freedom, provided your returned full seven-angle path satisfies every constraint.

Return a list or tuple of at most **10,000 seven-angle states** in radians. This final-path cap is separate from the search-tree size and query budget. Preserve the exact first and last requested states. Use finite bounded angles and no consecutive duplicates. If equal endpoints are valid, return a one-state path. Return None when search fails. Do not clip, wrap or silently repair native joint angles.

Call oracle.state_valid(q) for candidate states and oracle.edge_valid(a, b) for connections. Every call consumes one shared query, including rejection. Valid endpoints alone do not make a connection valid. Recover the path from your own parent links and recheck tree joins or shortcuts. The supplied evaluator checks the final route again.

Provided kinematics and world geometry

FK maps seven angles to the tool's world position and orientation. **IK** attempts one requested tool pose from a seed and returns candidate angles or None. Different seeds can produce different solutions. These helpers compute poses, not routes, and do not establish collision validity.

Call or field	Meaning
kinematics.fk(q)	ToolPose: position_m and unit quaternion_wxyz.
kinematics.ik(position, quaternion, seed)	Candidate seven-angle tuple or None. Check its state and connections.
kinematics.geometry(q)	Detached robot, cup, peer, bead and map collision surfaces in world metres.
context.nominal_obstacles	Named nominal obstacle boxes with lower_m and upper_m.
context stage fields	active_arm_id, motion_kind, source_support_id and destination_support_id. No intermediate posture.

The geometry query exposes actual collision-mesh vertices and triangle indices, exact box extents and sphere radii. It also identifies entity and link ownership. Map query padding is reported separately from nominal surfaces. The held cup follows the grasp transform while peers keep their current poses. The call changes no stored scene state and performs no physical step.

Use this geometry to compute your own clearance estimate or additional constraint. A tool-only distance may miss an elbow or cup collision. Sample points do not represent every point on a curved surface. Keep the supplied whole-robot state and edge checks. One geometry call shares the FK budget. Inspect remaining_fk_calls and remaining_ik_calls for optional queries.

Joint-space distances use radians. Physical clearance uses metres. context.edge_max_joint_delta_rad describes the supplied connection sampling. You may sample more finely or impose stricter constraints. Fewer returned waypoints cannot weaken final validation. Sampled checks do not prove continuous collision freedom.

4. Set Speed and Motion Limits

The supplied rest-to-rest quintic interpolator automatically accelerates from rest and decelerates to rest on every path edge. Commanded velocity and acceleration are zero at each waypoint. Choose paths and edge times in **student/joint_strategy.py** and speed scale in the selected **student/*_design.json**. You do not implement a separate braking routine or low-level controller. Actual joints follow through physics and can lag behind the command, including near a waypoint or support contact.

Set joint_speed_scale in (0, 1]. Each edge uses its analytic minimum duration divided by that scale. A smaller scale slows the path and changes acceleration. For explicit per-edge times, set scale to 1.0, implement choose_edge_durations_s and enable --edge-timing-hook. Return one positive simulated duration per edge, at least minimum_edge_duration_s(a, b, limits). Invalid times reject instead of being stretched.

Use these fixed instructor-chosen task limits when selecting times. They are the same for Lower, Moderate and Higher. Joint numbers follow the seven-angle path order. The supplied limits also contain inclusive joint-position bounds in radians.

Arm joints	Max. absolute velocity (rad/s)	Max. absolute acceleration (rad/s²)
1-4	1.0	2.0
5-7	1.4	2.8

The physical review checks actual velocity and acceleration against these limits. Actual acceleration is (v_after - v_before) / 0.0025 over one outer physics interval. Its absolute value is checked, so acceleration and deceleration can both reject a run. Actual joint position must track the command within 0.03 rad. Physics stays at 0.0025 s with 16 internal substeps.

CPU minimum times guarantee the interpolated command limits only. A legal command can still produce **control:acceleration_limit** in physical replay. On the same edge, doubling duration halves peak commanded velocity and quarters peak commanded acceleration. Actual physics need not follow that scaling, and a longer duration does not guarantee a pass.

Use the failed stage and tick in the supplied JSON records to locate the relevant saved edge. Increase its duration or revise your path, then create a fresh CPU export. No fixed slowdown factor is required. The run guide names the exact fields. Unnecessary waypoints add stops and can increase time. Tracking, contact and bead-retention checks still apply. There is no independent 8-degree spill gate.

Understand the two clocks

The simulated clock excludes the first 200 preparation ticks (0.5 s). It then includes physical approach, carry, handling, waits and required settling. Supported donor returns retrace their own path with zero physics ticks. This display operation checks contact at the final home pose only, without a swept collision or physical-return claim. The final carrier does not return. Deadlines of 40/85/105 s are inclusive.

Computer runtime is separate. CPU records split student search callbacks from remaining preparation/validation/export work. Default review guards are 1,800/3,600/5,400 s and sampled process VRAM is limited to 5 GB. These are operational guards, not estimates. Search time remains reported separately. GTX 1660 SUPER time and memory remain unqualified until measured on that hardware.

Measured Lower example on an **RTX 4090**: **20 min 17.289 s** for configuration, CUDA check, three tests, fresh CPU planning, full bead review and HTTPS registration. CPU planning took **22.308 s**. Native review took **19 min 40.741 s**, including preparation, physics, checks and recording. This one run is not a GTX 1660 estimate. Path, speed, cache state and hardware affect cost.

5. Test the Current Glass-Bead Stage

Work from your repository root in Windows Anaconda Prompt. Use the existing applied-programming-cuda environment. PyTorch 2.14.0+cu126 and Genesis 1.3.2 remain fixed. Save the instructor-issued token outside Git at %USERPROFILE%\applied-programming-team.token. Configure it once for this checkout, then check the environment and your implementation.

```
conda activate applied-programming-cuda
python -B project_run.py configure --token-file "%USERPROFILE%\applied-programming-team.token"
python -B project_run.py check --cuda --output output/environment.json
python -B project_run.py test
```

REPORTING_CONFIGURED confirms the local token-file setting. The environment check must report ENVIRONMENT_CUDA_SMOKE_PASS. Wait for guidance on a mismatch before changing packages. The unfinished planner has two path errors and a passing timing check. All three checks should pass after implementation. These tests do not establish map feasibility.

Use the development process in Figure 4. The physical replay modes below read the CPU plan.

Follow the exact commands in Test, replay and submit. Choose one complexity block, which sets LEVEL, MAP and a fresh RUN folder. Do not copy terminal prompts or previous printed output.

Stage	What it establishes
CPU plan	FK, geometric state/edge checks, joint bounds and analytic command timing. Output is saved_tier_mission.json. Actual motion and deadline success remain unproved.
Optional empty cup	Replays the same saved layout, initial poses, joints and times with only beads removed. Tests physical tracking, contact, handling and completion. No competition record.
Full bead review	Replays the same plan with four bead IDs. Checks actual motion limits, retention, prohibited contacts, tracking, handoffs, support and final rest. Only full bead attempts enter competition history.

No online RRT, IK or hidden speed adjustment occurs during replay. After changing student code or design, export a fresh CPU plan. Preserve old outputs. Actual loss stops a run. Beads are not regenerated or restored at a handoff.

Inspect your own planned motion

After a successful CPU export, make an optional viewer file:

```
python -B export_student_view.py ^
  --saved-tier "%RUN%/cpu/saved_tier_mission.json" ^
  --output "%RUN%/motion_preview.json"
```

Open **docs/map_viewer.html from your cloned repository** in a browser. Keep **motion_preview.js** beside it. Select the same complexity, choose **Preview your saved motion** and load motion_preview.json. Use **Stage**, **Frame** or **Play** to inspect seven joint angles, planned arm/cup geometry and exact handoff points. **Preview your design JSON** shows bases and handoff points before planning. No reference solution is preloaded.

The preview does not replan, retime or start Genesis. Its stage command time is not competition completion time. It shows planned geometry, not physical tracking, bead motion or a new collision certificate. Continue with physical replay.

6. Record Results and Compare Designs

Every completed full bead review automatically queues and uploads success or failure using your configured team token. CPU, empty-cup and short practice runs never enter competition history. Students need no ngrok or VPN installation.

The supplied project_run.py also reports failed check, test, plan and review commands to the instructor privately. Reports contain a bounded error summary, stage, settings, environment versions and source hashes. Student source code, credentials and full logs are not uploaded. Reporting does not change physics, results or scores and adds no student form.

Inspect REVIEW_SUPERVISOR.json for physical success. A full pass requires review_accepted, physical_transport_accepted and full_mission_passed all true. LEADERBOARD_STATUS.json reports upload separately: uploaded means acknowledged, pending means queued. scoring_enabled: true identifies an active rule. An upload acknowledgement is not a physics pass.

Diagnostic uploaded confirms an error report reached the instructor. Diagnostic pending means it remains local. After reconnecting, run **python -B project_run.py retry** to resend diagnostics and leaderboard records without rerunning Genesis. Keep the original output. The run guide gives recovery, history and selection commands.

Distance and time are equally weighted

D is actual cup-centre travel while held, summed over physical outer steps and stored in whole millimetres. It includes held waits/handling. It excludes unheld approach, released settling and the zero-time donor return. It is not joint-space distance or planned tool-path length.

T is the verified task-completion clock in 0.0025-second ticks. For one successful representative per team at the same complexity and active rule, normalize using that population's minima and maxima:

```
Dn = (D - Dmin) / (Dmax - Dmin)
Tn = (T - Tmin) / (Tmax - Tmin)
C  = 0.5 * Dn + 0.5 * Tn
```

Lower C ranks higher. A metric with no variation contributes zero for everyone. The other weight stays 0.5. Exact rational arithmetic determines ties. Equal composites share rank, for example 1, 1, 3. Computer speed and upload order do not break ties.

The live board uses each team's latest successful run. Failures stay in history without replacing that success. The official board uses each team's explicitly selected successful run. The instructor freezes selection for assessment. Ranges and composites can change when the comparison population changes.

Choose a reproducible final representative

Choose one successful acknowledged run under your active rule and commit and push its evaluated student files. The selection command checks every Python and JSON file under student/, including helper modules and unused tier designs, against the run snapshot. Use the full 40-character commit ID. Confirm status: selected and the intended run_id and commit_id in its response. A changed planner or design cannot be attached to an older result. Keep the matching code, settings and run evidence for your presentation.

Use failed phase, contact and retention records to revise layout, path or speed. Compare designs under stated conditions. Explain why a change helped or failed, and what remains uncertain. This interpretation belongs in the existing presentation, not a new verification report.

7. Assessment and Competition

For the current glass-bead release, let M be the base maximum: Lower 80, Moderate 90, Higher 100. The maximum is not an automatic award. Assess the submitted implementation and matching execution evidence using these components.

Component	Maximum	Assessed evidence
Path implementation	20% of M	State/edge selection, route recovery, search failure and query handling.
Layout / handoffs / geometry	20% of M	Permitted choices and use of clearance or constraints to address reachability and interference.
Speed design	10% of M	Valid scale or edge times connected to the chosen motion.
Valid completion	50% of M	Verified complete carries / required carries (1, 2 or 3).

A valid carry includes verified arrival, retained beads and supported release/rest. A partial path does not earn part of a carry. Later failure does not erase earlier independently verified carries. Unusable evidence remains unknown for review, not invented as a measured zero. Incomplete runs do not enter competition.

For multi-person teams, apply an individual code-contribution adjustment of -5 to +5, then clamp the base to [0, M]. Solo contribution adjustment is 0. Add competition and presentation adjustments afterwards, with a final floor of 0. Analysis is assessed through the presentation rather than another base category.

Example: a Moderate team earning 30 of 50 normalized design points and completing one of two valid carries has 90 x (30/100 + 0.5 x 1/2) = 49.5 base points before contribution adjustment.

Competition adjustment

Rank successful representatives by C within their own complexity. A solo entry counts as one team.

Complexity	Rank 1	Rank 2	Rank 3
Lower	+5	+3	+1
Moderate	+10	+5	+3
Higher	+15	+10	+5

Fourth place receives 0. If the group contains N >= 5 successful representatives, lower ranks receive:

```
A(C) = -N * (C - C4) / (Cmax - C4)
```

C4 is the composite of the fourth sorted entry, including ties. Cmax is the largest composite. If Cmax = C4, lower-rank adjustments are all zero. With four teams, fourth place also receives zero. Failed runs have no competition adjustment and are excluded from N.

For six teams with C4 = 0.50 and Cmax = 1.00, composites of 0.50, 0.75 and 1.00 give adjustments of 0, -3 and -6. See the scoring reference.

8. Present and Submit Your Work

Form your own team within the permitted complexity/team-size table. Create **one shared private repository** from the public course template for all team work and final submission. Add all members and the instructor account **SSUMechE** as collaborators. Each member must commit and push their own work using their own GitHub account. Keep design files, saved paths and motion previews private. The public leaderboard shares result summaries, not your layout or path.

One team PowerPoint

Prepare **2-3 common slides plus 3 slides per member**. The title slide is excluded. Each member presents their own named section. Teams of one, two and three therefore prepare 5-6, 8-9 and 11-12 content slides.

Common part. Explain the mission, planning method, constraints, route and speed choices. Show results and interpret improvements, tradeoffs and limitations. Include the shared private repository URL, final full 40-character commit ID, case/settings and reproduction commands.

Each member's three slides. Identify your name, role, edited files/functions and key changes. Explain how implementation, debugging or analysis affected the result. Use the actual commit and push history to explain your contribution. Link your pushed commits and relevant pull requests. Identify joint work and distinguish your contribution.

Presentation assessment

Presentation adds -10 to +10 after the base ceiling: team quality (-5 to +5) plus individual explanation (-5 to +5). Both components apply to solo entries. Intermediate values may be awarded. Actual code contribution is assessed separately.

Component	**-5**	**0**	**+5**
Shared team quality	Disjointed explanation or key claims lack evidence or contradict results.	Coherent mission, method and results with traceable code and records.	Clear connection between choices, comparable evidence, tradeoffs and limitations.
Individual explanation	Cannot identify or explain own changes and relevant planning logic.	Accurately explains role, changed code and observed effect with attribution.	Explains why work affected outcomes, interprets limits and identifies own part in joint work.

Week 15 deliverables

Retain the implemented planner and configuration, supplied check results and saved run records supporting the presentation. Submit one team PowerPoint. Include the repository reference in that deck. Submit the repository URL and final full 40-character commit ID in the LMS. Online leaderboard registration does not replace the presentation or LMS submission.

The project starts in Week 11 and culminates in Week 15 presentation assessment and final submission. The instructor announces the exact closure and freezes official representative selections. No extra student form, writing quota or verification report is required.

Optional preparation: MIT's RRT exercise explains single-tree RRT, bidirectional RRT and shortcutting. The Motion Planning chapter explains configuration-space search. Use these concepts with the provided project API. MIT's Docker/Drake environment is not required.

Updates and Recovery

Run maintenance from your repository root in Windows Anaconda Prompt with the existing applied-programming-cuda environment. Stop all project commands first, including direct debug, preview export and leaderboard commands.

The updater changes verified supplied files only. It preserves **student/**, output records and upload queues, credentials, unknown files and Git history. It does not install packages, commit changes, plan a path or run Genesis. PyTorch and Genesis remain fixed.

Inspect and apply an exact release

Set RELEASE to the exact tag announced by the instructor. This documentation release is shown as an example:

```
set "RELEASE=term-v3-material-table-20261114"
python -B project_run.py update check --release "%RELEASE%"
```

UPDATE_CHECK_PASS means the release and local supplied files passed checks. It does not apply files. Read the proposed changes, then apply that same tag:

```
python -B project_run.py update apply --release "%RELEASE%"
python -B project_run.py update status
```

UPDATE_APPLIED gives a backup_id. Keep that backup under output/project_updates/. **UPDATE_ALREADY_CURRENT** means no update was needed. **UPDATE_STATUS** reports local state and backups. Use update check to verify against the official release.

After an applied update, run the environment check and supplied tests. Review, commit and push the supplied-file changes through your normal team Git workflow. Create a **fresh CPU export** before the next physical replay. Preserve old runs and their original rule identities. Upload retry does not update code or relabel old records.

Resolve a refusal or roll back

UPDATE_REFUSED names the reason and affected paths. Local edits to supplied files or unknown-file collisions stop before overwrite. Keep the files for instructor review. Do not reset the repository. **UPDATE_BUSY** means another project command must finish first.

If an applied update causes a problem, stop project commands and replace BACKUP_ID with the ID from UPDATE_APPLIED or update status:

```
python -B project_run.py update rollback --backup BACKUP_ID
```

UPDATE_ROLLED_BACK confirms that supplied files were restored. Rollback preserves later student edits and existing results. It refuses to overwrite supplied files edited after the update. If you committed the update, commit the rollback as a new change. After interruption, run update status before resuming project work. If the main runner is unavailable during the first migration, recover offline with the retained bootstrap file: **python -B "%BOOT%\update_project.py" status**.

Earlier template and detailed commands

If your original template has no update command, use the one-time bootstrap in the run guide. It fetches the exact official tag and runs the updater from a fresh ignored output folder. Keep that file for rollback to the old runner. The guide gives exact environment/test commands, expected results and failure recovery.